

Computing Methods for Physics – 10 May 2023

Your exam material (code files, plots, etc.) must be submitted via google classroom by 13:00 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, setters/getters, unnecessary void functions, correct mathematical expression and physical units, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

The electric field generated at a point $\vec{r} = (x, y, z)$ by a set of N point-like charges is given by the superposition of the electric fields they generate individually. Denoting the electric charges with $\{q_1, \dots, q_N\}$ and the location of the i -th charge with $\vec{r}_i = (x_i, y_i, z_i)$, then in vacuum

$$\vec{E}(\vec{r}) = \sum_{i=1}^N \vec{E}_i = \sum_{i=1}^N \frac{1}{4\pi\epsilon_0} \frac{q_i}{|\vec{r} - \vec{r}_i|^3} (\vec{r} - \vec{r}_i), \quad (1)$$

where $\epsilon_0 = 8.854 \cdot 10^{-12} \text{ C}^2 \text{ N}^{-1} \text{ m}^{-2}$.

1. Design and implement a class `PointCharge` to represent a single point-like charge characterized by charge and location vector values.
2. Design and implement a class `PointChargeDist` to represent a system of point-like charges. This class must rely on `std::vector` and the `PointCharge` class. The `PointChargeDist` regular constructor must take as an argument the name of a `txt` file where the user specifies the charge and location of the charges in the system.
3. Provide methods to add and to remove the i -th charge from the set of charges.
4. Provide getters for the number of charges and a method to print to screen the properties of all charges in the system.
5. Provide a method to evaluate the components of $\vec{E}$ given $\vec{r}$, according to Eq. (1).
6. Write an application `app.cpp` to showcase the methods of your classes.

Part 2 – Python

The motion of a particle with charge q and mass m in a magnetic field $\vec{B}$ is regulated by the following system of first order ordinary differential equations:

$$\begin{aligned}\frac{d\vec{x}}{dt} &= \vec{v} \\ \frac{d\vec{v}}{dt} &= \frac{q}{m}(\vec{v} \times \vec{B}),\end{aligned}$$

where $\vec{x}$ and $\vec{v}$ are the position and velocity vectors of the particle, respectively. Use a Python notebook or Python scripts to complete the following tasks.

1. Use `scipy.integrate.odeint` to simulate the motion of a particle, with $\vec{x}(t = 0)$, $\vec{v}(t = 0)$, q , m , and $\vec{B}$ provided by the user.
2. Plot the results of your simulation.
3. Simulate the motion of $N = 10$ particles, with $\vec{B}$, a single initial position and a single initial velocity (both to be used for all particles) provided by the user. The masses and charges must instead be chosen randomly: allow the user to pick between Gaussian and Uniform distributions (one choice per physical parameter), and to provide the parameters of the chosen distributions.
4. Plot the results of your simulation.
5. Store the results of this simulation in a `pickle` file.

In doing so, exploit dictionaries and NumPy arrays as much as possible.